```
puts "Before: obj = #{obj}"
data = YAML.dump(obj)
obj = YAML.load(data, permitted_classes: [Special])
puts "After: obj = #{obj}"
```

produces:

```
Before: obj = Hello there World
After: obj = Hello  World
```

For security purposes, YAML.load takes a list of classes that you expect to find in the incoming YAML text and that you are willing to create instances of based on the contents of the YAML text.

We can take a look at what YAML creates as the serialized form of the object—it's pretty simple:

```
obj = Special.new("Hello", "there", "World")
puts YAML.dump(obj)
```

produces:

```
--- !ruby/object:Special
precious: World
valuable: Hello
```

JSON is another very commonly used interchange format, and Ruby also has a JSON module that provides JSON loading and dumping. But Ruby's JSON module does not, by itself, parse Ruby objects into Ruby objects, it converts Ruby objects to hash data. If we want to restrict the attributes, or convert to and from actual objects, typically we need to do this ourselves:

ospace/json_load.rb

```
require "json"

class Special
  def initialize(valuable, volatile, precious)
    @valuable = valuable
    @volatile = volatile
    @precious = precious
  end

  def self.from_json(json_string)
    result = JSON.parse(json_string)
    Special.new(result["valuable"], nil, result["precious"])
  end

  def to_json
    JSON.dump(
      {
        precious: @precious,
        valuable: @valuable
      }
    )
  end

  def to_s
    "#{@valuable} #{@volatile} #{@precious}"
  end
```